

JobFlow 概要设计说明书

1. 系统概述

1.1 项目背景

JobFlow 是一款面向求职者的桌面端智能求职辅助系统，主要用于解决求职过程中岗位发现效率低、岗位筛选困难、公司信息分散、投递记录难以管理以及面试经验难以沉淀等问题。

传统求职方式下，用户需要分别进入多个招聘平台，根据关键词搜索岗位，并逐条阅读岗位描述，再结合自身经历判断是否适合投递。当需要同时关注大量岗位时，用户需要投入大量时间进行重复的信息检索和人工筛选。同时，不同平台的岗位信息结构并不统一，相同岗位可能在多个平台重复出现，进一步增加了岗位筛选和管理成本。

在完成岗位投递后，用户还需要面对面试准备和复盘问题。面试过程中出现的技术问题、项目问题以及自身无法回答的问题通常分散在聊天记录、笔记或个人记忆中，难以形成持续积累的面试知识库。

JobFlow 将用户简历、求职偏好以及招聘平台岗位信息进行统一管理，并利用 Agent 对简历和岗位进行理解、匹配和分析，在此基础上为用户推荐更加符合个人条件的岗位。同时，系统提供公司背景信息分析、岗位投递、投递状态跟踪以及面试问题总结与复盘等功能，从而形成完整的求职辅助流程。

1.2 系统目标

JobFlow 的总体目标是构建一个以用户求职画像为基础、以 Agent 为核心智能能力、以多平台岗位检索为主要数据来源的智能求职工作台。

系统主要实现以下目标：

1. 将用户非结构化简历转换为结构化求职画像；
2. 根据用户简历及求职偏好从多个招聘平台发现潜在岗位；
3. 对不同来源岗位进行统一建模、清洗和去重；
4. 对岗位与用户背景进行多维度匹配分析；
5. 对岗位进行推荐排序并解释推荐原因；
6. 聚合公开企业信息，辅助用户判断公司背景；
7. 在用户确认的基础上辅助完成岗位投递；
8. 对投递后的岗位进行统一状态管理和跟踪；
9. 对面试过程中的问题和用户回答情况进行总结与复盘；
10. 为用户沉淀可持续使用的面试知识和个人能力画像；
11. 提供兼容多种大模型服务协议的统一模型接入能力。

1.3 设计原则

模块化原则。

按照业务职责对系统进行模块划分，降低模块之间的耦合程度。

Agent 与业务解耦原则。

Agent 负责复杂的信息理解和决策辅助，业务系统负责数据管理、任务调度和流程控制，避免将全部业务逻辑放入 Prompt 中。

平台适配原则。
针对不同招聘平台建立独立的数据采集适配层，使新增招聘平台时无需修改核心业务逻辑。

模型适配原则。
大模型服务通过统一抽象层接入，不直接将业务 Agent 与某一个厂商模型绑定，使用户可以根据自身需求配置不同模型服务。

异步任务原则。
对于岗位搜索、简历解析、公司调查、面试分析和自动投递等耗时任务采用异步执行方式，避免阻塞桌面端 UI。

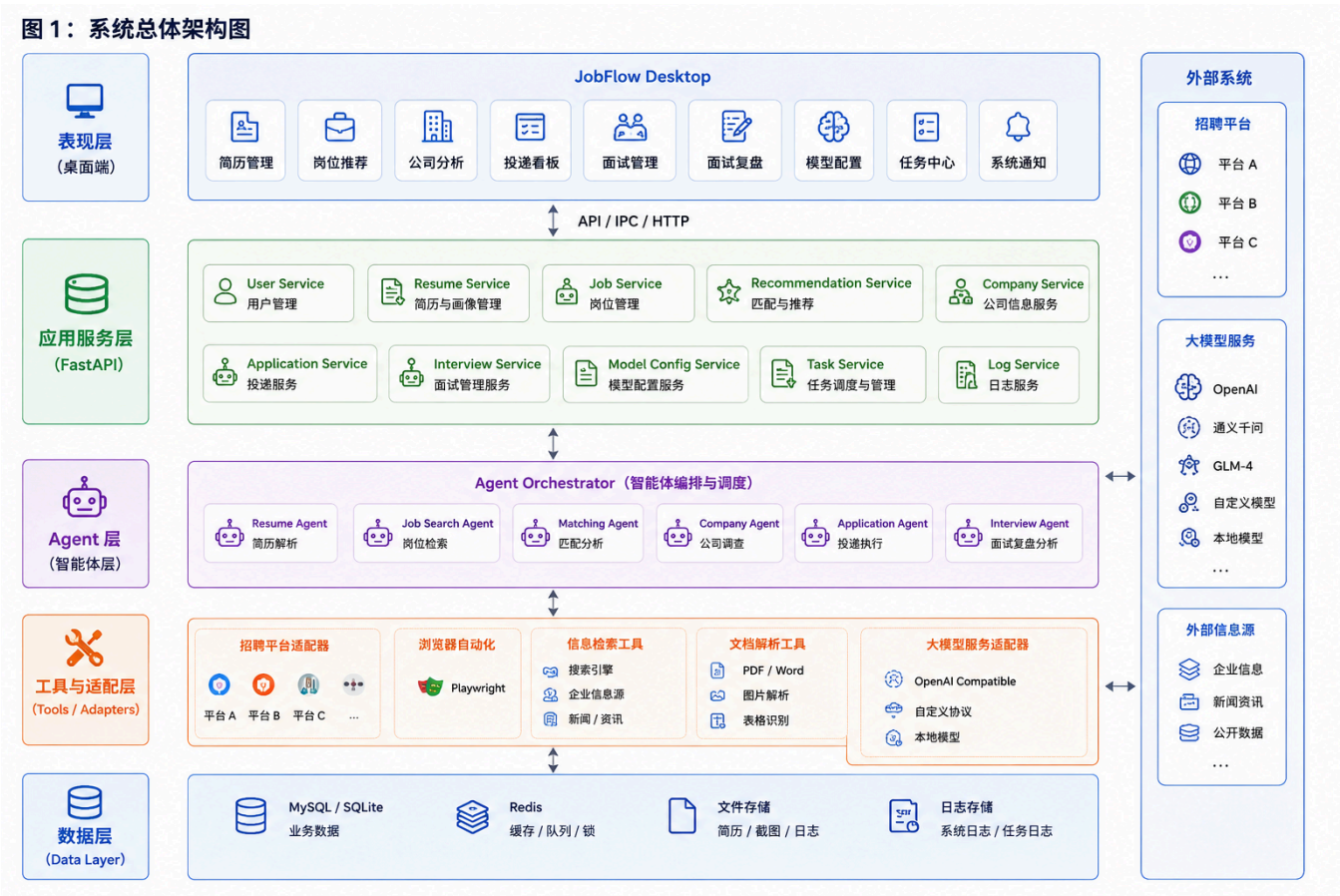
人工可接管原则。
对于验证码、登录失效、页面状态不确定以及需要用户确认的操作，允许任务暂停并交由用户处理。

数据安全原则。
简历、联系方式、招聘平台登录信息以及 API Key 等敏感数据应得到保护，并对敏感日志进行脱敏。

2. 系统总体架构

2.1 总体架构

JobFlow 采用桌面端 + 业务服务 + Agent + 工具适配层 + 数据层的分层架构。



系统中增加统一的 **大模型服务适配器**，将 Agent 与具体大模型服务解耦。

2.2 桌面端设计

JobFlow 采用桌面应用形式，前端主要负责：

- 用户界面展示;
- 简历文件选择;
- 求职偏好配置;
- 岗位推荐展示;
- 岗位详情查看;
- 公司信息展示;
- 投递看板;
- 面试日程;
- 面试复盘;
- Agent 任务状态展示;
- 大模型配置;
- 系统通知。

桌面端前端采用 Vue3 + Element Plus 实现。

桌面容器可以采用 Electron 或 Tauri，具体选型在技术设计阶段进一步确定。

2.3 服务端设计

服务端采用 FastAPI 构建业务 API，负责：

- 用户数据管理;
- 简历数据管理;
- 岗位数据管理;
- 推荐结果管理;
- 投递记录管理;
- 面试记录管理;
- Agent 任务创建;
- 异步任务调度;
- 模型配置管理;
- 外部服务调用;
- 系统日志管理。

3. 系统功能模块设计

3.1 用户画像模块

用户画像模块负责维护用户的基本信息、简历信息以及求职偏好。

主要功能包括：

- 简历上传;

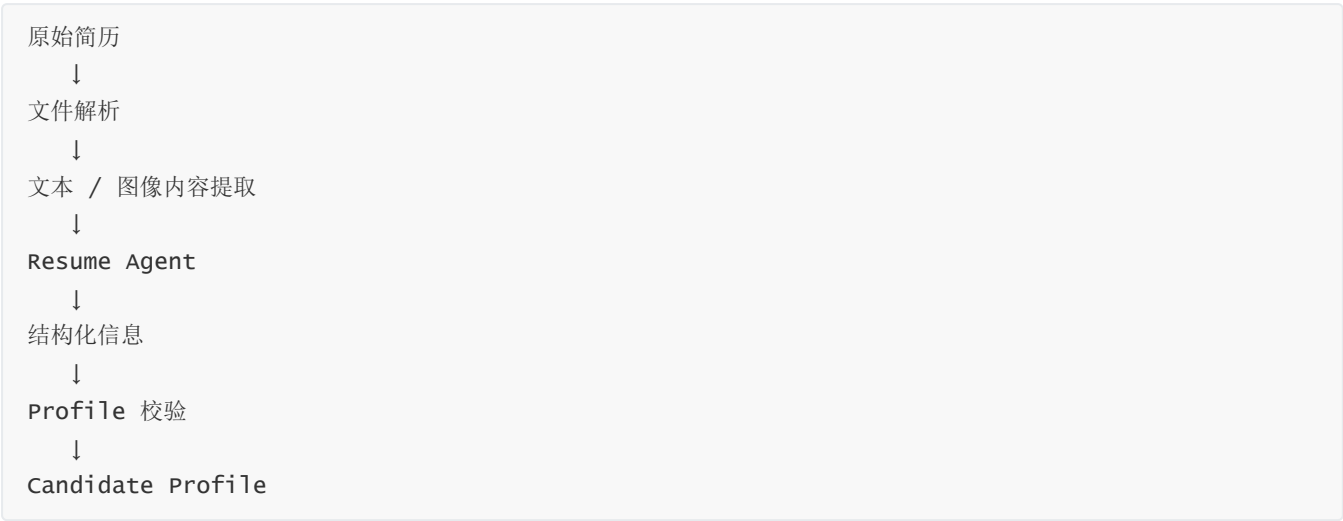
- 简历解析;
- Candidate Profile 生成;
- 求职偏好设置;
- 用户信息修改。

该模块是整个系统的数据入口，为后续岗位搜索、匹配以及面试分析提供统一的用户画像。

3.2 简历解析模块

简历解析模块负责将 PDF、DOCX 或图片格式的简历转换为结构化信息。

处理流程：



模块输出主要包括：

- 基本信息;
- 教育经历;
- 工作经历;
- 项目经历;
- 技能;
- 证书;
- 获奖情况;
- 其他经历。

解析结果需要经过格式校验，以减少 LLM 输出异常对系统后续业务造成的影响。

3.3 岗位采集模块

岗位采集模块负责从多个招聘平台获取岗位数据。

为了适应不同平台之间的页面结构和字段差异，系统采用 Adapter 设计。

```
Job Source
|
├─ Platform Adapter A
├─ Platform Adapter B
├─ Platform Adapter C
└─ Other Adapter
```

每一个 Adapter 负责完成对应平台的：

- 岗位搜索；
- 岗位详情获取；
- 公司信息获取；
- 原始数据转换。

核心业务系统只接收标准化 Job 对象，而无需了解具体平台的页面结构。

3.4 岗位数据标准化模块

来自不同平台的岗位数据格式存在差异，需要在进入核心业务系统之前进行标准化。

标准化后的 Job 对象包括：

```
Job
├─ job_id
├─ title
├─ company_id
├─ city
├─ salary
├─ education
├─ experience
├─ job_type
├─ responsibilities
├─ requirements
├─ publish_time
├─ source
└─ source_url
```

系统同时保留部分原始岗位数据，以支持后续调试和数据追溯。

3.5 岗位去重模块

相同岗位可能在多个招聘平台重复发布，因此需要进行岗位去重。

系统可以综合：

- 公司名称；
- 岗位名称；
- 城市；

- 岗位描述;
- 招聘要求;
- 发布时间;

判断两个岗位是否可能为同一岗位。

去重后的岗位作为用户最终看到的推荐对象。

3.6 岗位匹配与推荐模块

该模块负责分析用户与岗位之间的匹配程度，并根据多个因素对岗位进行排序。

匹配因素包括：

- 技能匹配;
- 项目 / 工作经历匹配;
- 教育背景匹配;
- 岗位硬性要求;
- 用户薪资和城市偏好;
- 工作类型偏好。

输出包括：

- 综合匹配度;
- 推荐等级;
- 匹配优势;
- 不匹配项;
- 潜在风险;
- 推荐理由。

系统不仅提供评分，还需要说明：

为什么推荐该岗位，以及用户与该岗位之间存在哪些明显差距。

3.7 公司信息分析模块

用户查看岗位时，可以主动请求公司分析。

系统可以基于公开信息整理：

- 公司基本信息;
- 公司规模;
- 所属行业;
- 成立时间;
- 融资情况;
- 上市情况;

- 主要业务；
- 公司官网信息；
- 公开招聘情况。

系统进一步提供风险提示。

风险提示应建立在可验证的信息基础上，避免将 Agent 主观推断直接作为事实。

3.8 投递模块

投递模块负责将用户选中的岗位转化为实际投递任务。

主要包括：

- 手动投递记录；
- 自动化投递；
- 投递任务管理；
- 投递结果保存；
- 投递异常处理。

自动投递使用 Playwright 作为浏览器自动化工具。

3.9 投递跟踪模块

系统提供统一投递状态：

```
graph TD; A[待投递] --> B[已投递]; B --> C[等待回复]; C --> D[笔试]; D --> E[一面]; E --> F[二面]; F --> G[HR 面]; G --> H[offer];
```

同时允许：

- 已拒绝；
- 已结束；
- 主动放弃。

系统通过 Kanban 对这些状态进行可视化管理。

3.10 面试管理与复盘模块

面试管理模块负责记录和整理用户的面试过程。

面试记录

用户可以记录：

- 公司；
- 岗位；
- 面试轮次；
- 面试时间；
- 面试方式；
- 面试官；
- 面试备注；
- 面试结果。

面试问题记录

用户可以手动记录面试中出现的问题，例如：

问题：

Redis 的持久化机制有哪些？

我的回答：

只能回答出 **RDB** 和 **AOF** 的基本概念。

结果：

回答不完整。

面试总结 Agent

系统可以利用 Interview Agent 对面试记录进行分析，输出：

- 本次面试主要考察方向；
- 面试问题分类；
- 已掌握的问题；
- 回答不完整的问题；
- 完全不会的问题；
- 需要复习的知识点；
- 潜在薄弱项。

例如：

本次面试主要涉及：

Java 基础

★★★★★

Spring

★★★★☆

Redis

★★☆☆☆

MySQL

★★★☆☆

系统进一步可以形成个人的**面试知识库**：

面试问题



问题分类



用户回答



Agent 分析



知识点总结



用户薄弱项



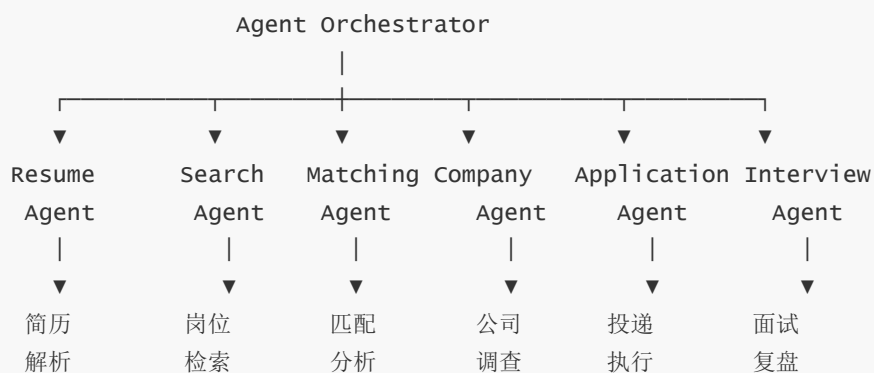
后续复习

这样面试记录不再只是历史日志，而可以逐渐形成用户自己的面试能力画像。

4. Agent 系统设计

4.1 Agent 总体架构

JobFlow 采用多个专业 Agent + Agent Orchestrator 的结构。



Agent Orchestrator 负责：

- Agent 调度；
 - Agent 之间的数据传递；
 - 工具调用；
 - 任务状态控制；
 - 异常处理；
 - 结果汇总。
-

4.2 Resume Agent

输入

- 原始简历文件；
- 文本或图像解析结果。

输出

Candidate Profile。

核心职责

- 提取个人基本信息；
- 提取教育经历；
- 提取项目与工作经历；
- 提取技能；
- 统一字段格式；
- 对缺失信息进行标记。

Resume Agent 不应主动编造简历中不存在的信息。

4.3 Job Search Agent

输入

- Candidate Profile；
- 用户求职偏好。

输出

Search Task。

核心职责

- 理解用户求职目标；

- 生成搜索关键词;
 - 分解搜索条件;
 - 调用招聘平台 Adapter;
 - 汇总岗位结果;
 - 根据搜索结果进行必要的查询扩展。
-

4.4 Matching Agent

输入

Candidate Profile + Job + User Preference。

输出

Match Result。

主要包括:

- 综合匹配度;
 - 技能匹配;
 - 经历匹配;
 - 学历匹配;
 - 硬性条件匹配;
 - 偏好匹配;
 - 优势;
 - 不足;
 - 推荐等级;
 - 推荐理由。
-

4.5 Company Agent

输入

Company。

输出

Company Profile + Risk Information。

主要职责:

- 收集公开公司信息;
- 对公司信息进行结构化整理;
- 检查招聘主体与公司名称;

- 分析公开企业背景；
 - 输出信息来源及风险提示。
-

4.6 Application Agent

输入

Job + User Profile + Resume。

输出

Application Result。

主要职责：

- 创建投递任务；
 - 调用浏览器自动化工具；
 - 执行页面操作；
 - 监控操作结果；
 - 保存投递状态；
 - 处理可恢复异常；
 - 在必要时暂停并请求用户接管。
-

4.7 Interview Agent

Interview Agent 负责处理面试过程中的结构化和非结构化信息。

主要输入：

- 面试记录；
- 用户记录的问题；
- 用户回答；
- 面试岗位 JD；
- 用户 Candidate Profile。

输出：

- 面试问题分类；
- 回答质量分析；
- 未掌握知识点；
- 面试总结；
- 后续复习建议；
- 用户面试能力画像。

后续可以进一步接入历史面试数据，实现：

5. 大模型接入与协议兼容设计

5.1 设计目标

JobFlow 不直接将 Agent 与某一家大模型厂商绑定。

系统应提供统一的大模型配置和调用抽象，使用户可以根据自身需求选择不同模型服务。

用户可以在桌面端配置：

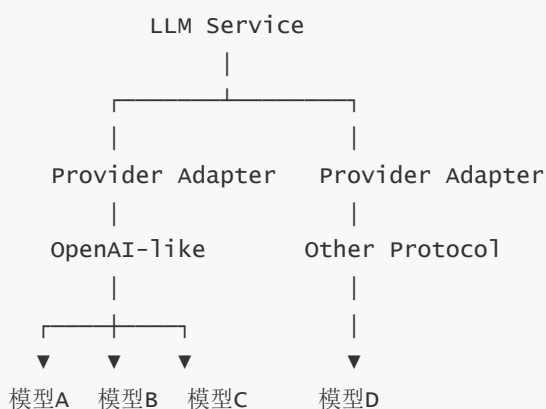
- Provider;
- Base URL;
- API Key;
- Model Name;
- 其他模型参数。

典型配置形式为：

```
Provider: 自定义 / OpenAI Compatible
Base URL: https://example.com/v1
API Key: *****
Model: xxx-model
```

5.2 统一模型适配层

系统增加 **LLM Provider Adapter**：



Agent 只调用统一的 LLM Service，而不直接调用具体厂商 SDK。

例如：



这样可以降低后续更换模型的成本。

5.3 OpenAI Compatible 协议

对于遵循 OpenAI Compatible API 规范的第三方模型服务，系统优先通过统一接口进行接入。

用户只需要配置：

```
Base URL
API Key
Model
```

即可接入不同服务。

这种方式可以兼容大量提供标准化 Chat Completions / Responses 类 API 的模型服务，而无需为每个服务编写独立业务逻辑。

对于具有独立 API 协议的模型服务，可以进一步实现专用 Provider Adapter。

5.4 模型配置管理

系统应允许用户保存多个模型配置，例如：

```
模型配置
├──
├── 默认模型
├── 低成本模型
├── 高性能模型
└── 本地模型
```

不同 Agent 可以根据任务特点选择不同模型。

例如：

Resume Agent

→ 普通模型

Matching Agent

→ 高能力模型

Company Agent

→ 支持联网/工具调用模型

Interview Agent

→ 高能力模型

后续可以增加统一的 Model Routing 能力，根据任务复杂度自动选择模型。

5.5 API Key 安全

API Key 属于敏感凭证，不应直接保存到普通数据库字段或普通日志中。

桌面端应优先利用操作系统提供的安全凭证存储机制，或者采用加密方式保存。

日志中不得记录完整 API Key。

6. 核心业务流程设计

6.1 用户画像建立流程

用户上传简历

↓

文件解析

↓

Resume Agent

↓

生成 Candidate Profile

↓

用户检查

↓

用户修正

↓

保存 Profile

6.2 岗位搜索流程

用户设置求职要求

↓

Job Search Agent

↓

生成 Search Task

↓

调用多个 Platform Adapter



获取原始岗位



数据标准化



岗位去重



保存 Job



进入匹配流程

6.3 岗位匹配与推荐流程

Candidate Profile



Job



Matching Agent



Match Result



Ranking Service



推荐岗位列表



用户查看

Ranking Service 可以综合：

- 匹配度；
- 用户偏好；
- 岗位时效性；
- 公司因素；
- 历史行为。

6.4 公司分析流程

用户打开岗位详情



获取 Company



Company Agent



检索公开信息



信息整理与交叉验证



Company Profile



风险提示



展示给用户

6.5 投递流程

用户选择岗位



确认投递



创建 Application Task



Application Agent



Playwright



打开招聘平台



定位岗位



填写信息



上传简历



提交



记录投递结果

6.6 面试复盘流程

用户记录面试



输入面试问题 / 回答



Interview Agent



问题分类



回答分析



薄弱知识点识别



面试总结



写入个人面试知识库



形成后续复习依据

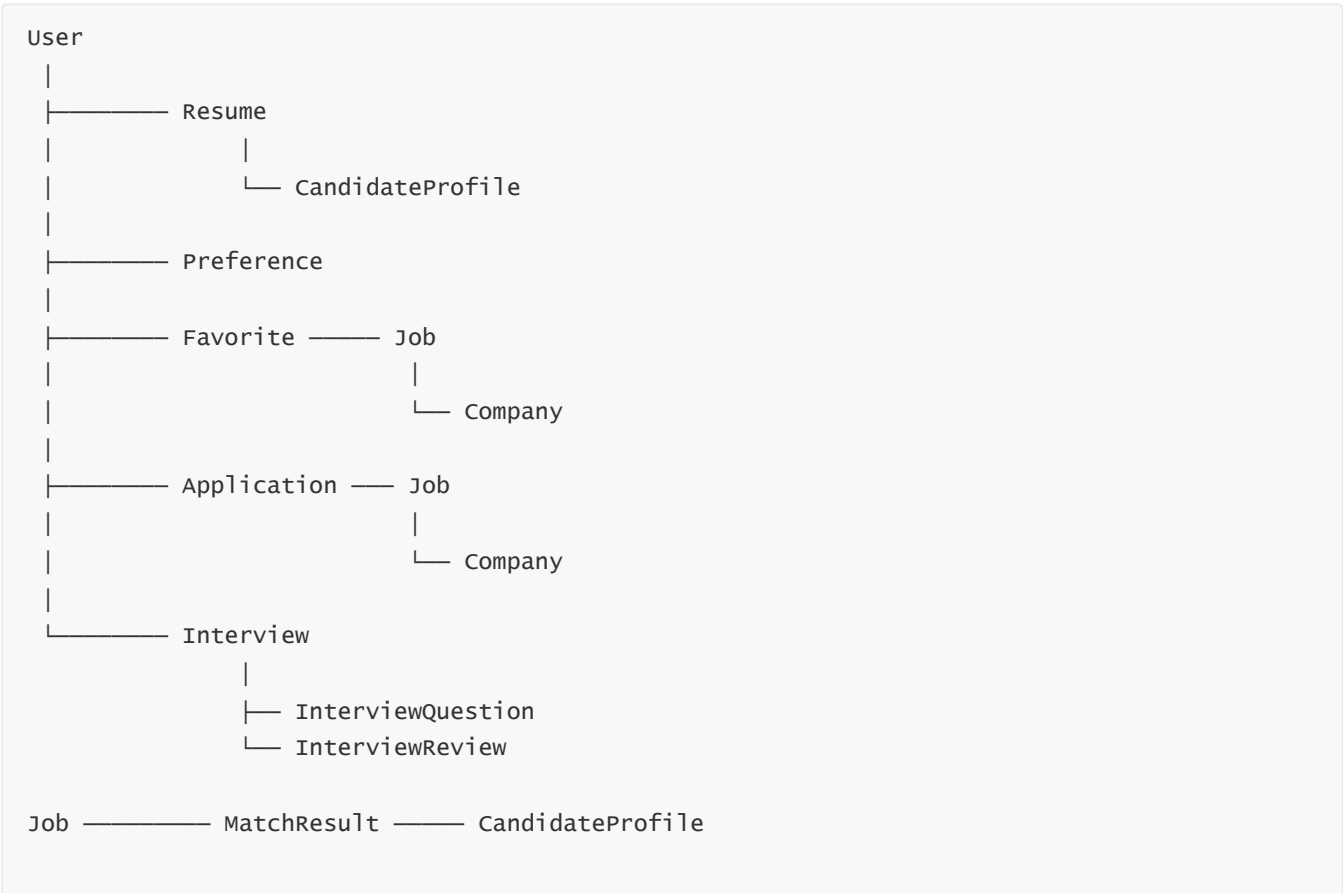
7. 数据架构设计

7.1 核心实体

系统核心实体包括：

```
User
Resume
CandidateProfile
Preference
Job
Company
JobSource
MatchResult
Favorite
Application
Interview
InterviewQuestion
InterviewReview
ModelProvider
ModelConfig
AgentTask
AgentLog
```

7.2 核心实体关系



User — ModelConfig — ModelProvider

AgentTask — AgentLog

7.3 数据存储设计

MySQL 主要保存：

- 用户；
- 简历；
- 用户画像；
- 求职偏好；
- 岗位；
- 公司；
- 匹配结果；
- 收藏；
- 投递；
- 面试记录；
- 面试问题；
- 面试复盘；
- 模型配置元信息。

Redis 主要用于：

- 缓存；
- Agent 临时状态；
- 异步任务状态；
- 分布式锁；
- 短期数据。

简历原始文件可以采用本地文件存储或对象存储，并在数据库中保存文件元信息。

API Key 等敏感凭证不建议直接以明文形式存入普通业务数据库。

8. 任务与状态管理

8.1 异步任务

以下操作设计为异步任务：

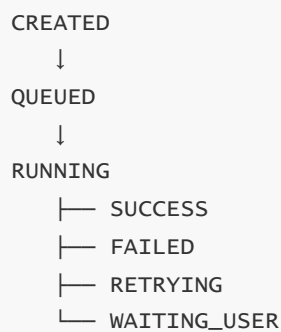
- 简历解析；
- 多平台岗位搜索；
- 岗位批量分析；

- 公司信息调查;
- 自动投递;
- 面试总结;
- 定时岗位刷新。

系统使用 Celery 负责任务执行，Redis 负责消息队列和任务状态管理。

8.2 Agent 任务状态

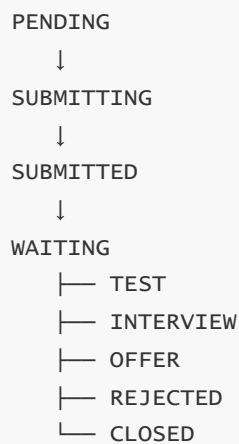
每一个 Agent Task 均具有明确状态：



用户可以在桌面端查看任务状态。

8.3 投递状态

投递业务状态可以设计为：



8.4 面试状态

面试记录可以进一步采用状态管理：

SCHEDULED
↓
IN_PROGRESS
↓
COMPLETED
↓
REVIEWED

面试完成后，可以自动创建 Interview Review 任务，由 Interview Agent 生成总结。

9. 异常处理设计

9.1 Agent 异常

包括：

- LLM 请求失败；
- 输出格式错误；
- JSON 解析失败；
- 信息不足；
- 工具调用失败；
- 模型服务不可用。

处理方式：

Agent Error
↓
格式校验
↓
可重试？
├── 是
│ ↓
│ Retry
└── 否
 ↓
 Task Failed

模型请求失败时，可以根据配置进一步执行：

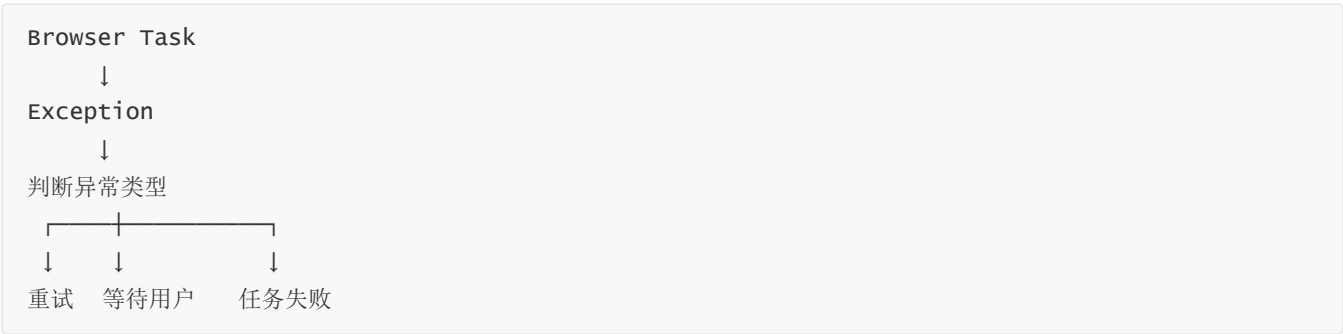
主模型失败
↓
Retry
↓
备用模型
↓
仍失败
↓
通知用户

9.2 浏览器自动化异常

包括：

- 网络异常；
- 页面加载失败；
- 登录失效；
- 页面结构变化；
- 元素不存在；
- 验证码；
- 操作结果无法确认。

处理方式：



对于验证码或需要用户判断的场景，任务进入 `WAITING_USER` 状态。

10. 外部系统设计

10.1 招聘平台

系统通过 Platform Adapter 对接招聘平台。

每个平台 Adapter 对外提供统一接口：

```
search_jobs()
get_job_detail()
get_company_info()
```

核心服务只依赖统一接口，不直接依赖具体平台实现。

10.2 大语言模型服务

系统通过统一 LLM Service 接入不同模型服务。

基础调用链：

```
Agent
↓
LLM Service
↓
Provider Adapter
↓
Model Config
↓
Base URL / API Key / Model
↓
具体模型服务
```

该设计允许在不修改 Agent 业务逻辑的情况下切换大模型服务。

10.3 浏览器自动化

Playwright 作为 Browser Tool，为 Application Agent 提供：

- 浏览器启动；
- 页面访问；
- 元素定位；
- 表单填写；
- 文件上传；
- 页面截图；
- 操作结果获取。

Playwright 不负责判断“应该投递什么岗位”，只负责执行 Application Agent 确定的浏览器操作。

11. 安全与隐私设计

由于 JobFlow 涉及完整个人简历、联系方式、招聘平台登录状态以及大模型 API Key，系统需要重点保护用户隐私。

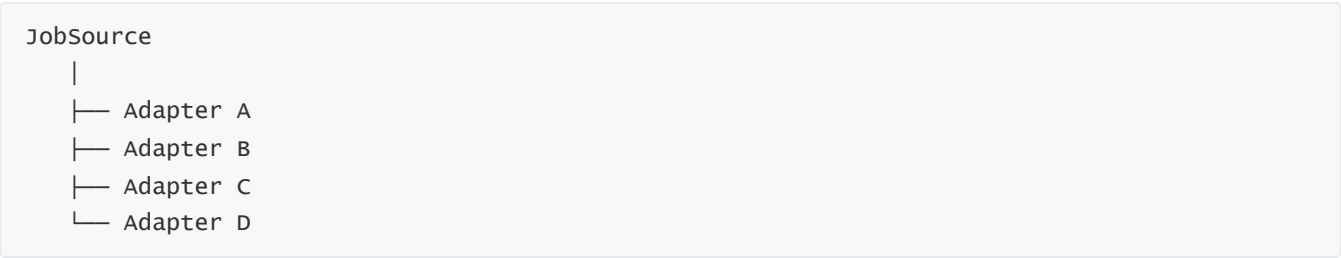
主要措施包括：

1. 简历数据进行权限隔离；
 2. 敏感信息不直接写入普通日志；
 3. API Key 不硬编码；
 4. API Key 优先使用系统安全凭证存储；
 5. 招聘平台登录信息采用安全方式保存；
 6. Agent 日志进行脱敏处理；
 7. 用户能够删除简历及相关数据；
 8. 自动投递操作必须在用户授权范围内执行。
-

12. 可扩展性设计

12.1 招聘平台扩展

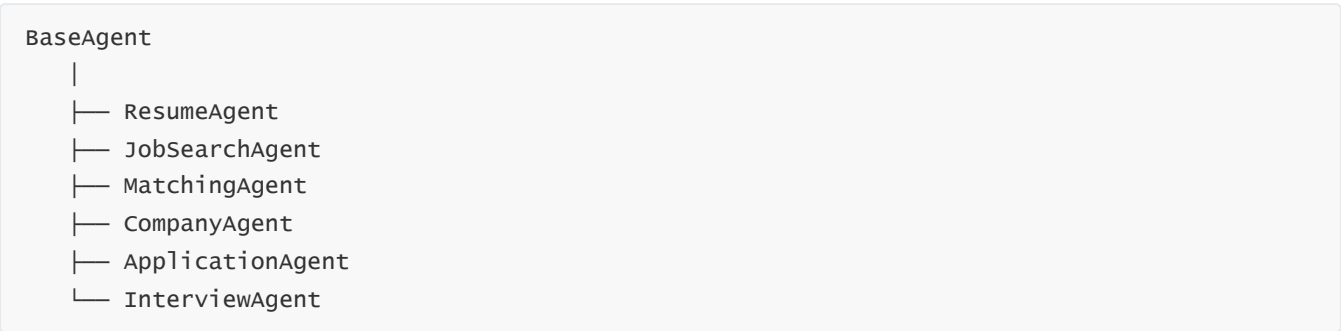
采用 Adapter 机制：



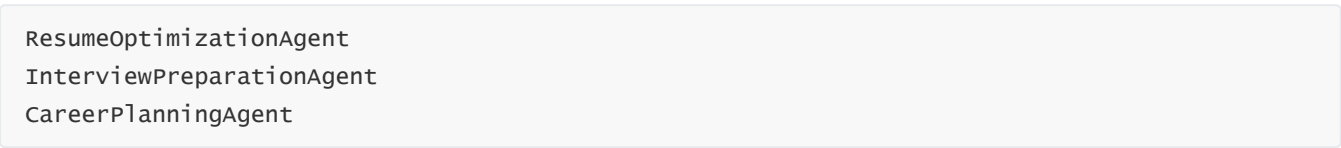
新增平台时只需要实现对应 Adapter。

12.2 Agent 扩展

通过统一 Agent Interface 管理不同 Agent：

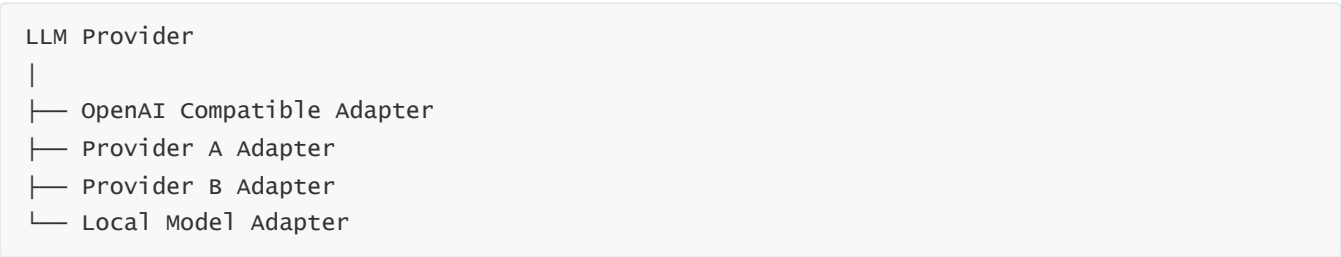


后续可以继续增加：



12.3 模型服务扩展

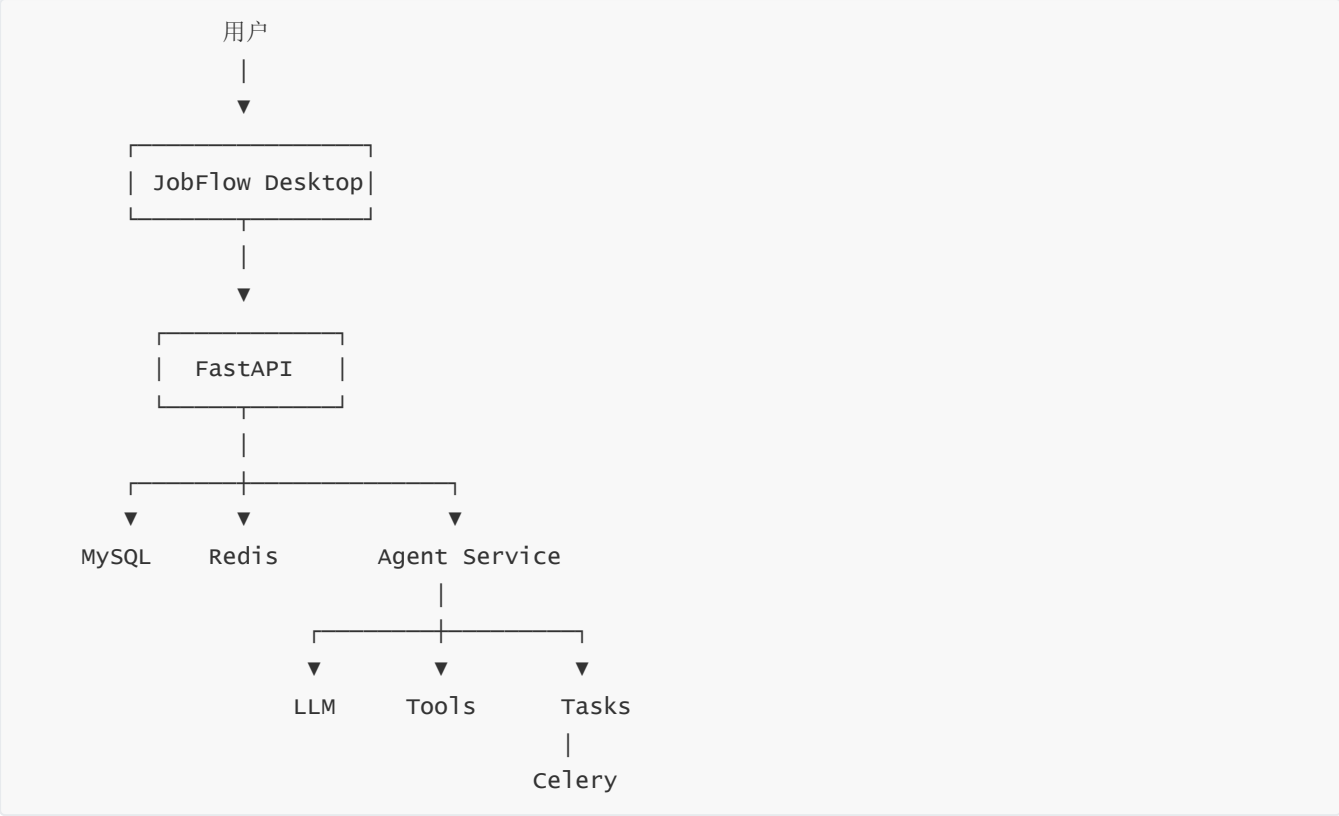
模型层通过 Provider Adapter 进行扩展：



新增模型服务不会影响上层 Agent 的业务逻辑。

13. 系统部署结构

初步部署结构如下：



系统可以通过 Docker 对后端服务进行容器化部署。

14. 概要设计总结

JobFlow 的整体设计采用**桌面端 + 业务服务 + Agent + 工具适配层 + 数据层**的分层架构。

在业务层面，以岗位作为核心业务对象，形成：

用户画像 → 岗位发现 → 岗位匹配 → 推荐排序 → 公司调查 → 用户决策 → 投递 → 跟踪 → 面试复盘
的完整业务闭环。

在智能层面，通过多个专业 Agent 分工处理不同任务：

Resume Agent 负责理解用户，**Job Search Agent** 负责发现岗位，**Matching Agent** 负责评估岗位，**Company Agent** 负责调查企业，**Application Agent** 负责执行投递，**Interview Agent** 负责总结和分析面试。

在模型接入层面，通过统一的 **LLM Service + Provider Adapter** 实现多模型、多协议兼容。用户可以自主填写 **API Key**、**Base URL** 和 **Model**，从而接入不同的 OpenAI Compatible 服务以及其他支持独立协议的模型服务。

在工程层面，通过 Adapter 机制解决多平台接入问题，通过 Redis + Celery 解决耗时任务和异步执行问题，通过状态机和 Human-in-the-loop 机制解决自动化过程中的异常和不确定性，通过 Interview Agent 将一次性的面试记录进一步转化为可以持续积累的个人面试知识。

因此，JobFlow 的核心不只是“帮助用户投递简历”，而是构建一个围绕求职全过程运行的智能辅助系统：

理解用户 → 主动找岗 → 判断岗位 → 调查公司 → 辅助投递 → 跟踪结果 → 总结面试 → 沉淀能力。